

Machine Learning Through the Lens of Representation

Mohammad Mahdi Nasimi

Contents

1	Why This Course Exists	2
1.1	What Is Machine Learning?	2
1.2	The Cheap Habit	3
1.3	What the Notebook Showed	4
1.4	Our Approach	4
1.5	The Modern Engineer's Mandate	5
1.6	Prerequisites and Logistics	5
1.7	Summary	6
1.8	Bridge to Session 1	7
1.9	Exercises (Ungraded)	7

1 Why This Course Exists

Session 0

One-line motivation. You already know how to call `sklearn`. This session is about why calling `sklearn` is not the same as understanding machine learning—and about what we intend to teach you instead.

This session has a companion Jupyter notebook. Run it before reading these notes, or alongside them. The notebook contains a full demonstration of the failure mode these notes are about: a conventional ML pipeline applied to a projectile problem, followed by a debrief and a reconstruction. We will not repeat the code here. We will explain what it means.

1.1 What Is Machine Learning?

Machine learning is the process of learning useful structure from data. We observe some inputs and outputs, and we want a program that maps new inputs to correct outputs.

Classical Software Engineering vs. Machine Learning

In classical software engineering, a human expert encodes domain knowledge into a deterministic algorithm. The engineer designs a function $f : \mathcal{X} \rightarrow \mathcal{Y}$ that maps inputs to outputs using explicit logic. The computer executes it. This works beautifully for well-defined problems: computing taxes, sorting a list, calculating a trajectory. It fails when the rules are unknown, too numerous, or too subtle to articulate.

Machine learning *inverts* the process. Instead of writing the function f by hand, we:

1. Choose a family of candidate functions—a *hypothesis space* $\mathcal{F} = \{f_\theta \mid \theta \in \Theta\}$.
2. Collect examples—a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ of input–output pairs.
3. Define a loss function $\mathcal{L}$ that measures how wrong a candidate is.
4. Search for the parameters θ^* that make the function as correct as possible.

In compact form:

$$\text{ML: } (\mathcal{D}, \mathcal{F}, \mathcal{L}) \xrightarrow{\text{Optimizer}} \theta^* \implies f_{\theta^*}(x) \approx y \quad (1)$$

Analogy. Classical programming is like giving someone a recipe. Machine learning is like giving them a kitchen, ingredients, and a way to measure whether the food tastes good—and letting them figure out the recipe themselves.

Newton Was Doing Machine Learning

Before machine learning had a name, scientists were already doing it.

Between 1609 and 1619, Kepler published three empirical laws of planetary motion. The third is the cleanest: the square of a planet’s orbital period is proportional to the cube of its semi-major axis:

$$T^2 \propto a^3.$$

Kepler did not know why. He had found a pattern in Tycho Brahe’s data. This is descriptive learning. It is what a regression model does.

Newton, in 1687, proposed a mechanism: an inverse-square force law,

$$F \propto \frac{1}{r^2}.$$

From this single law, plus his three laws of motion, he *derived* Kepler’s third law. It was not a fit. It was a consequence. And when he tested the same law against the Moon’s orbit, the tides, and the precession of the equinoxes—phenomena he had not used to derive it—it worked.

Newton had three things a modern regressor does not.

First, **a tiny hypothesis space**. He considered algebraic force laws of the form $F \propto r^n$ for a small set of n . The whole space could be written on a blackboard.

Second, **an independent verification**. He did not test on the same data he used to derive the law. He tested on the Moon, on tides, on precession. These were chosen by reasoning about what a gravitational law *should* affect.

Third, **a mechanism**. The law was not just accurate. It was explanatory. It connected planets to falling apples in a single framework.

Machine learning is not a departure from classical science. It is the automation of hypothesis selection at scale. But automation is a tool, not a substitute for understanding. Newton automated nothing, but he understood everything he did. We can automate everything and understand nothing.

1.2 The Cheap Habit

The companion notebook contains a full demonstration of a projectile-range problem treated as a conventional machine learning task. The instructor tries linear regression, polynomial features, ridge regularization, a model zoo, and finally a Random Forest. Every model achieves acceptable cross-validation scores. The Random Forest achieves a test R^2 of 0.68. The demo looks like a success.

Then the notebook steps out of the role and asks what actually happened. The answer is: nothing was understood, and the model would fail catastrophically outside the range where it was tested.

Let us name the practice.

Definition 1.1: The Cheap Habit

The **cheap habit** is the practice of selecting, tuning, and deploying models by numerical score, without understanding what the model assumes, what structure the phenomenon has, or whether the two are compatible.

The cheap habit is not stupidity. It is a natural response to a real problem. Modern ML libraries make it extremely easy to run dozens of models in a single afternoon. The APIs are clean, the documentation is good, and the results often look acceptable. Under time pressure—and engineering is always under time pressure—the cheap habit is the path of least resistance.

It is also a trap. It fails in four predictable ways.

It cannot detect silent failures. A model can have excellent test metrics and still be wrong. It can latch onto a spurious feature, fail to extrapolate, or collapse under a distribution shift that no held-out sample captured. Metrics cannot see any of these failures. Only understanding can.

It cannot be defended. When a model fails in production, the review committee will ask why this model, why this hypothesis class, what it assumes, and why the assumption is justified. “It had the best cross-validation score” is not an acceptable answer.

It cannot be improved. If you do not know why a model works, you cannot make it work better. You can only try more models. This is called hyperparameter tuning, and it is the cheapest possible form of engineering: no insight, no principle, only more compute.

It does not transfer. A model tuned by the cheap habit is tuned for one dataset. Change the sensor, the operating range, or the operating conditions, and the tuning is worthless.

The cheap habit produces models that work by accident. Accidental models work until they do not, and when they fail, they fail silently and catastrophically.

1.3 What the Notebook Showed

The projectile problem in the notebook is the simplest possible illustration of the cheap habit’s failure.

The true range is $R(\theta) = (v^2/g) \sin(2\theta)$: a sinusoid of period 180° in θ . This is the shape of the phenomenon. It is fixed by physics, not by the data. No amount of looking at the measurements in $[10^\circ, 45^\circ]$ can reveal that the range will fall back down at 80° ; only the physics tells us.

Every model in the demo assumed a shape—linear, polynomial, piecewise-constant, arbitrary smooth—that did not match the sine. Each could fit the training window. None could extrapolate. The Random Forest’s apparent success was a coincidence of the specific test range we happened to choose.

The notebook then shows what happens when we start from the shape instead: choose a hypothesis class of sinusoids of period 180° , using the pair $(\sin 2\theta, \cos 2\theta)$ as the minimal basis. The same data, the same tools, and a completely different outcome.

The fix was not a bigger model, more data, or better regularization. The fix was to write down the shape first, then choose a hypothesis class that respects it.

This is the entire course in miniature. Every session that follows will elaborate it.

1.4 Our Approach

Machine learning is the process of learning useful structure from data. Representation is the central lens through which that structure becomes visible, usable, and connected to decision-making.

We will not teach you a catalog of algorithms. We will not teach you how to call `sklearn`. We will not teach you to trust cross-validation scores. We will teach you to reason about what a model *assumes*, what it *preserves*, and what it *discards*. We will teach you to look at a problem, ask what shape the phenomenon has, and ask whether your hypothesis class matches that shape.

What This Course Is

- A first-principles reconstruction of machine learning as geometry and representation.
- A guided journey from raw, misleading coordinates to meaningful, decision-ready structure.
- A training ground for engineers who must diagnose systems, not just build them.
- A course where every algorithm earns its place by solving a representational problem the previous one could not.

What This Course Is Not

- A survey of algorithms.
- A preparation for a Kaggle competition.
- A tutorial in calling library functions.
- A statistics course, though it uses statistical concepts.

The Boundaries

To keep the course focused, we draw strict boundaries on Day 1. We will not cover, except in passing:

- **Bayesian methods.** When uncertainty, belief updating, and small data are the primary problems, Bayesian methods are required. That is a different course.
- **Reinforcement learning.** When an agent takes actions over time to maximize a reward, RL is required. That is a different course.
- **Causal inference.** When the goal is to intervene in a system rather than just observe it, causal methods are required. That is a different course.

By bounding the course, we can fully commit to the geometric viewpoint without ignoring the realities of data. We are not ignoring statistics. We are spatializing it. Noise is a geometric perturbation. Variance is the instability of a decision boundary. Everything is grounded in how physical data spaces warp and behave.

1.5 The Modern Engineer's Mandate

With LLMs and powerful hardware commoditizing the ability to write boilerplate code and brute-force hyperparameter searches, the value of the modern engineer has shifted.

LLMs cannot detect silent representational invalidity. They can write the code that produced the failed demo. They cannot tell you that the demo was the wrong demo.

The purpose of this course is to train diagnosticians who can formulate the problem correctly, choose the right coordinate representation, and recognize when a system is fundamentally broken before it ships to hardware.

1.6 Prerequisites and Logistics

What You Need to Know

This course assumes the following, all at the working level:

- **Linear algebra.** Vectors, matrices, matrix multiplication, change of basis, eigenvalues and eigenvectors.
- **Calculus.** Partial derivatives, gradients, chain rule, basic optimization.
- **Probability and statistics.** Distributions, expectation, variance, conditional probability, basic estimation.
- **Python.** You should be able to read and write Python, use NumPy and Matplotlib, and run Jupyter notebooks.

We will not re-teach any of this. If a gap appears in your background, address it independently. The course moves quickly and assumes fluency.

The Delivery Stack

- **LaTeX lecture notes.** Rigorous mathematical spine. Released weekly.
- **Slide presentations with digital pen.** Dynamic geometric drawing during lecture.
- **Live Jupyter notebooks.** Code written in class, in front of you, translating ideas into runnable form.
- **GitHub repository.** All materials, all notebooks, all ungraded homework. Released weekly.
- **Telegram group.** Asynchronous discussion, questions, and support between sessions.

The Format

- **Zero stakes.** No grades, no formal assessment.
- **Volunteer enrollment.** You are here because you paid for this course and chose to be here.
- **5 weeks, 10 sessions.** Two 1.5-hour sessions every Thursday, back to back.
- **Ungraded homework.** Optional but strongly encouraged. It is where depth lives.

Because there are no grades, the only thing this course can offer you is understanding. Everything is structured around that fact.

1.7 Summary

1. The companion notebook demonstrated the cheap habit: a full ML pipeline that produced a working model with no understanding.
2. The cheap habit is not stupidity; it is a natural response to an easy tool. It is also the trap this course exists to escape.
3. We replace the cheap habit with a representational lens: shape, hypothesis class, and the match between them.
4. The course follows a narrative wire. Every algorithm will be motivated by the failure of the previous one.

5. The course ends with an engineering post-mortem, where the tools of the course are applied to real system failures.

1.8 Bridge to Session 1

Everything so far has been motivation.

The course begins now. The question is not “how do we build a model?”—you already know how. The question is:

How do we know which representation is right for our problem?

The answer begins in Session 1 with a broken toy: a cantilever beam whose raw coordinates are misleading, and whose physical coordinates are not obvious until you already understand the physics. It is the simplest possible example of everything the notebook showed going wrong. We will spend the next ten sessions learning why.

1.9 Exercises (Ungraded)

1. **Reproduce the demo.** Run the companion notebook. Confirm the numbers. Then answer: at what angle does the Random Forest prediction diverge most from the truth? Why?
2. **Extrapolate further.** Rerun the demo with the test range extended to $[10^\circ, 150^\circ]$. What does the Random Forest predict at 150° ? What is the true range? What does this tell you about the model’s inductive bias?
3. **Write down the shape.** The true projectile range is $R(\theta) = v^2 \sin(2\theta)/g$. Write one sentence describing its shape. What is the smallest hypothesis class you can think of that contains it?
4. **Find the spurious feature.** Construct a dataset where a model with excellent cross-validation accuracy fails catastrophically on a small, physically realistic shift in the input distribution. Describe the shift. Describe why no metric could have caught it.
5. **Reflection.** Recall the last machine learning model you built or used. Could you explain, in one paragraph, what its inductive bias is and why it matches the phenomenon? If not, what would you need to learn to be able to?